

Code review — cpp-initiere

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 2 — 2026-09-30 — sha `d9080ae`

Compilare:  `main.cpp` trece, 2 warning-uri · **Configurare:**  `cmake` refuză din prima linie (a doua oară) · **Rulare:**  26 funcții din `functii.h`, 8 exerciții din `exercitii-while.h`, 12 teme din `operatii-conditionale.h` · **Reproduce:** 15 bug-uri

Ai făcut de la runda 1 două lucruri mari. Primul: **ai spart cele 455 de linii din `main.cpp` în trei fișiere** — asta era M5, cea mai grea recomandare a rundei 1. Al doilea: **ai intrat în funcții** și ai scris 14 dintr-o suflare, cu reutilizare peste tot — `contorCifreImpare` cheamă celelalte două, `palindromNumar` cheamă `rasturnatNumar`, `cifraDeControl` cheamă `sumaCifrelor`. Reutilizarea aia e semnul că ai înțeles **la ce sunt bune** funcțiile, nu doar cum se scriu. Între timp ai dus și temele 29–53: parcări, colete, bagaje, locuri în sală, abonamente, curse de taxi. Vreo 25 de exerciții, majoritatea corecte, unele chiar elegante (`(minute + 59) / 60` ca să rotunjești ora în sus e un truc de om care a înțeles împărțirea întreagă).

Și chiar în timp ce scriam review-ul ăsta ai mai pushat **12 funcții** (`d9080ae`). Le-am luat și pe alea — review-ul e pe `d9080ae`, nu pe ce era dimineață. Două lucruri de acolo merită spuse tare, fiindcă sunt primele de felul lor:

- **`isDistincte` e corectă**. Și nu e evident de ce: tai numărul pe măsură ce-l parcurgi, deci ai putea pierde perechea. Nu se întâmplă, fiindcă atunci când ajungi la o cifră, toate copiile ei rămase sunt încă în `numar` — iar cele deja tăiate te-au văzut pe tine când au trecut. Verificat pe `1213`, `1123`, `1231`, `112`, `121`, `1122`: toate corecte.
- **`cifreDiferite` are `if (numar == 0) return 1;`** — prima gardă de zero pe care o scrii singur, fără să ți-o ceară nimeni. Ține minte gestul: B10 de mai jos e exact el, lipsă din alte șapte locuri.

Ai și răspuns în scris la Q&A — ești singurul care o face. Două din răspunsurile tale explică exact ce s-a întâmplat în runda asta, așa că le vei regăsi citate mai jos.

Și acum partea grea. **Din cele 5 🟠 ale runde 1, ai închis unul.** `tema12` e reparată corect. `tema26` — care e aceeași problemă, și pe care review-ul îți cerea explicit s-o reparați în aceeași propoziție — e neatinsă, literă cu literă. La fel B4, B5 și B1.

Nu cred că e neglijență, și îți spun de ce: **răspunsul tău la întrebarea 1.** Te-am întrebat care e mai periculoasă, cea care tace sau cea care minte. Ai răspuns „mai periculoasă este cea care tace, deoarece nu ne putem da seama de eroare”. Ai reparat-o pe cea care ți se părea periculoasă și ai lăsat-o pe cealaltă. Logica ta a fost consecventă — doar că răspunsul e pe dos, și B4 îți arată de ce.

Firul roșu al runde

Runda asta are **o singură temă**, și o vei recunoaște: *construiești o ramificare care nu acoperă toate cazurile*. În runda 1 erau trei apariții. Acum sunt șapte, în cinci forme diferite:

formă	unde	ce pățește utilizatorul
<code>else</code> cu etichetă care minte	<code>tema26</code>	„Numerele sunt egale” pentru 5 5 3
<code>cout</code> pus într-o singură ramură	<code>tema35</code>	15 minute la parc → ecran gol
<code>if</code> imbricat pe nivelul greșit	<code>tema32</code>	cere 310 lei, primește 300
<code>else if</code> acolo unde trebuiau <code>if</code> -uri	<code>tema33</code>	pune 65 de bani, i se dă restul din una singură
validare fără <code>else</code>	<code>tema41</code>	„Valoare invalidă” și o factură de -3 lei
<code>if</code> -uri independente unde trebuia <code>else if</code>	<code>tema31</code> , <code>tema10</code>	afișează de trei ori, sau deloc

Nu sunt șapte greșeli. E **una singură**, pe care o faci de șapte ori. De-aia mai jos le-am pus pe toate la un loc: dacă înțelegi mecanismul o dată, cazi toate șapte într-o după-amiază.

🟠 Critice — partea A: ramificări care nu acoperă toate cazurile

B1 — `tema35` nu afișează NIMIC pentru 15 minute la parc

`operatii-conditionale.h:629-652`

CUM E ACUM: rulat cu 15 :

Introduceti minutele dorite:

Atât. Nicio eroare, niciun preț. Cu 90 merge — „Total de plata: 8 lei”.

```
else {  
    int pret;  
    if ( minute <= 30 ) {  
        pret = 4;  
    }  
    else {  
        pret = 4 + ((minute - 30 + 29) / 30) * 2;  
        if (pret > 20) { pret = 20; }  
        cout << "Total de plata: " << pret << " lei" << endl;  
    }  
}
```

DE CE: calculul e corect — pentru 15 minute, `pret` chiar devine 4 . Problema e **unde stă `cout`** : l-ai pus înăuntrul lui `else` , nu după `if/else` . Ce e scris într-o ramură se execută doar când ajungi pe ramura aia; ramura `minute <= 30` calculează prețul și iese fără să spună nimic.

Acoladele nu sunt decorațiune, sunt un gard: totul ce e între ele se întâmplă doar pe drumul ăla.

`cout` -ul tău trebuie să fie pe **drumul comun**, adică după ce cele două ramuri se reunesc.

Și uite dovada că știi asta: `tema34` , `tema36` și `tema37` sunt exact aceeași problemă cu alte cifre, și în toate trei ai pus `cout` -ul afară, corect. `tema35` e singura din cele patru care a scăpat.

FIX: scoate `cout` -ul din `else` , la nivelul lui `int pret;` . Și mută și plafonul (`if (pret > 20)`) acolo, ca la `tema34` .

B2 — `tema32` : bancomatul îți dă 300 de lei când ceri 310

`operatii-conditionale.h:522-554`

CUM E ACUM: rulat cu 310 :

3 bancnote de 100

Lipsește bancnota de 10. Cu `780` merge perfect — 7 de 100, 1 de 50, 3 de 10.

```
if (bac100) {
    cout << bac100 << " bancnote de 100" << endl;
}
if (bac50) {
    cout << bac50 << " bancnote de 50" << endl;
    if (bac10) {
        cout << bac10 << " bancnote de 10" << endl;
    }
}
```

DE CE: cele trei bancnote sunt **independente** — faptul că ai zero de 50 nu spune nimic despre câte ai de 10. Tu ai pus `if (bac10)` **înăuntrul** lui `if (bac50)`, iar asta schimbă sensul: nu mai înseamnă „dacă ai bancnote de 10”, ci „dacă ai bancnote de 10 **și** ai și de 50”. Pe 310, `bac50` e `0`, deci întreg blocul e sărit — cu tot cu bancnota de 10 dinăuntru.

Imbricarea e o conjuncție ascunsă. Când scrii un `if` în alt `if`, spui „și”. Îl folosești corect la [tema42](#) (clasa biletului, apoi greutatea — acolo a doua întrebare chiar depinde de prima). Aici nu depinde nimic de nimic, deci cele trei `if` -uri trebuie să stea unul lângă altul.

Și partea urâtă: pe `780` merge. Ai testat cu un număr care are bancnote de 50 și ai plecat liniștit. **Un test care trece nu dovedește că programul e corect — dovedește doar că drumul ăla merge.**

FIX: scoate `if (bac10)` afară, frate cu celelalte două:

```
if (bac100) { cout << bac100 << " bancnote de 100" << endl; }
if (bac50)  { cout << bac50  << " bancnote de 50"  << endl; }
if (bac10)  { cout << bac10  << " bancnote de 10"  << endl; }
```

(și „bancnote”, nu „bacnote” — apare de trei ori)

B3 — [tema33](#) : automatul de cafea dă restul dintr-o singură monedă

`operatii-conditionale.h:557-590`

CUM E ACUM: pun `65` de bani (adică 50 + 10 + 5):

1 monede de 50

Automatul îmi oprește 15 bani. Codul:

```
if ( a != 0 ) {  
    cout << "Nu pot da restul exact !" << endl;  
}  
  
else if ( monede50 ) { cout << monede50 << " monede de 50" << endl; }  
else if ( moneda10 ) { cout << moneda10 << " monede de 10" << endl; }  
else if ( moneda5 ) { cout << moneda5 << " monede de 5" << endl; }
```

DE CE: e aceeași greșeală ca la B2, în cealaltă formă. `else if` înseamnă „sau”: lanțul se oprește la prima ramură adevărată și nu se mai uită la restul. Asta e exact ce vrei când întrebările se exclud între ele — un om are ori sub 7 ani, ori între 7 și 17, ori peste, niciodată două deodată. Dar cele trei monede **nu se exclud**: 65 de bani înseamnă și o monedă de 50, și una de 10, și una de 5, toate în același timp.

Regula de care să te agăți: `else if` pentru întrebări care se exclud, `if` -uri separate pentru lucruri care se adună. La restul de la automat se adună, deci `if` -uri separate — la fel ca la B2.

+ tot aici, `operatii-conditionale.h:585`: ramura `else if (a == sold)` cu „Suma exacta, cafea buna!” **nu poate fi atinsă niciodată**. Ca să ajungi la ea trebuie ca primul `if` să fi fost fals, adică `a <= 0 || a > sold`; dar `a == sold` cere exact `a > 0 && a <= sold`. Cele două condiții nu pot fi adevărate deodată, deci mesajul ăla e cod mort. Îl scrii o dată și nu-l vede nimeni niciodată.

FIX: cele trei `else if` devin trei `if`. Verificarea `a != 0` („nu pot da restul exact”) rămâne separat, înaintea lor.

B4 — tema26 tot minte pe `5 5 3` (*restanță din runda 1, B3*)

`operatii-conditionale.h:420-437`

CUM E ACUM: rulat cu `a=5, b=5, c=3`:

```
Numerele sunt egale
```

3 nu e egal cu 5. Codul e neschimbat față de runda 1.

DE CE: ai răspuns că e mai periculoasă cea care tace. E invers, și uite mecanismul.

Un program care tace pe `5 5 3` te forțează să te uiți la el: apeși Enter, nu apare nimic, îți dai seama în două secunde că s-a rupt ceva. Un program care scrie „Numerele sunt egale” **arată exact ca unul care funcționează**. N-ai niciun motiv să-l verifici. Dacă în locul lui `cout` ar fi fost o decizie — cine ia

bursa, ce preț se afișează, ce notă intră în catalog — greșeala ar fi plecat mai departe, cu tot cu aerul ei de răspuns corect.

Un bug care se vede costă un minut. Un bug care minte costă cât timp trece până îl găsește altcineva.

Tăcerea e un simptom; minciuna e o boală.

Cauza e cea din runda 1: `else` nu înseamnă „toate egale”, înseamnă doar „niciuna de mai sus n-a fost adevărată”. I-ai lipit o etichetă pe care n-o merită.

Și aici e partea care mă face să cred că știi deja: la `operatii-conditionale.h:1064` ai scris `testtema12()` — **varianta corectă**, cu `a == b && b == c` la început, `>=` în toate ramurile, `else` curat la final. Ai scris soluția bună. Ai lăsat-o alături și n-ai atins-o pe cea greșită. (Vezi M3 — se repetă.)

FIX: copiază forma din `testtema12` peste `tema26`, apoi șterge `testtema12`. O problemă = o funcție.

B5 — tema41 : „Valoare invalida” și, imediat sub ea, o factură de -3 lei

`operatii-conditionale.h:783-803`

CUM E ACUM: rulat cu `-5` kWh:

```
Valoare invalida
De plata: -3 lei si 0 bani
```

Și cu `0`:

```
Consum 0
De plata: 0 lei si 0 bani
```

```
if ( k < 0 ) { cout << "Valoare invalida" << endl; }
if ( k == 0 ) { cout << " Consum 0 " << endl; }
int cost;
if ( k <= 100 ) { cost = k * 60; }
else { cost = 100 * 60 + (k - 100) * 85; }
cout << "De plata: " << lei << " lei si " << bani << " bani" << endl;
```

DE CE: un `if` fără `else` **nu oprește nimic** — spune ceva și execuția merge mai departe pe același drum. Tu ai crezut că „Valoare invalida” încheie discuția; pentru compilator e doar o linie tipărită, după care urmează calculul, ca și cum nu s-ar fi întâmplat nimic. Cu `k = -5`, `cost` devine `-300`, iar `-300 / 100` dă `-3`.

O validare trebuie să **excludă** restul, nu doar să-l comenteze. Asta se face fie punând tot calculul pe `else`, fie oprind funcția pe loc.

Și iarăși: știi regula. `tema34`, `tema40`, `tema42`, `tema44 – tema48` și `tema51` au toate validarea pe `if` și calculul pe `else`, corect. `tema41` e singura scăpată.

FIX:

```
if ( k < 0 ) {
    cout << "Valoare invalida" << endl;
}
else {
    int cost;
    ...
    cout << "De plata: " << lei << " lei si " << bani << " bani" << endl;
}
```

🔴 Critice — partea B: funcții și bucle

B6 — `ex32` blochează programul la infinit

`exercitii-while.h:460-467`

CUM E ACUM: rulat cu `n = 5`, programul scrie prompt-ul și apoi nu mai face nimic. Nu se termină, nu dă eroare, nu poate fi oprit decât cu Ctrl+C. L-am omorât după 3 secunde.

```
void ex32 () {
    int n;
    cin >> n;
    while ( n != 0 ) {
    }
}
```

DE CE: un `while` reevaluează condiția după fiecare trecere prin corp. Corpul tău e gol, deci `n` rămâne `5` la nesfârșit și condiția `n != 0` rămâne adevărată la nesfârșit. Nu e o eroare de sintaxă — e

cod perfect legal, și de-aia compilatorul tace. În toate celelalte bucle ale tale există o linie care schimbă variabila din condiție: `n/=10` , `contor++` , `cin >> n` . Aici lipsește, și cu ea dispare singurul motiv pentru care o buclă se oprește vreodată.

Întrebarea adevărată e alta: **de ce n-ai observat?** Pentru că `main.cpp` nu include `exercitii-while.h` — vezi M1. Cele 468 de linii de acolo nu ajung la compilator și n-au fost rulate de când le-ai scris. Un `while` gol poate sta acolo la nesfârșit fără să dea vreun semn.

FIX: exercițiul e neterminat — ori îl scrii, ori îl ștergi. Nu-l lăsa pe jumătate: codul nerulat arată identic cu codul corect.

B7 — media notelor pierde zecimalele, acum în DOUĂ locuri (*restanță din runda 1, B4*)

`operatii-conditionale.h:190` și `exercitii-while.h:347`

CUM E ACUM: `tema4` cu notele `9, 10, 10` → `Media notelor este :9` . Și `ex24` , scrisă **după** runda 1, cu `9 10 10 0` → `Media este: 9` . Media adevărată e 9.67 în amândouă.

```
int medianotelor = (nota1 + nota2 + nota3) / 3;    // tema4
double media = double (suma / numar);              // ex24
```

DE CE: iată ce ai scris tu la întrebarea 2: „*deoarece programul nu citește variabila double, ci prima dată face împărțirea.*” Țasta e răspunsul corect, cuvânt cu cuvânt.

Și apoi ai scris `ex24` . `double (suma / numar)` face exact ce ai descris: mai întâi se evaluează parantezele — `29 / 3` cu doi `int` , adică împărțire întreagă, rezultat `9` , restul aruncat. Abia pe `9` -ul ăla gata rotunjit se aplică conversia la `double` . Obții `9.0` . Conversia nu poate recupera zecimalele, pentru că **ele nu s-au calculat niciodată.**

E mai interesant decât o simplă greșeală: știi regula în cuvinte, dar n-o recunoști încă în cod. `double(...)` pus pe dinafară *arată* ca o reparație — mâna a făcut gestul corect în locul greșit.

FIX: conversia trebuie să atingă **un operand**, nu rezultatul:

```
double medianotelor = (nota1 + nota2 + nota3) / 3.0; // tema4
double media = (double) suma / numar;                // ex24
```

B8 — `ProdusulCifrelorPare` întoarce `1` când nu există nicio cifră pară

`functii.h:181-191`

CUM E ACUM:

```
ProdusulCifrelorPare(3468) -> 192   corect
ProdusulCifrelorPare(135)  -> 1     gresit
ProdusulCifrelorPare(13579) -> 1     gresit
ProdusulCifrelorPare(7)    -> 1     gresit
```

135 nu are nicio cifră pară. Funcția răspunde **1** — un număr care arată ca un rezultat.

DE CE: pornești cu `produsul = 1` pentru că **1** e elementul neutru al înmulțirii: primul `produsul * cifra` trebuie să dea chiar `cifra`. Corect. Problema e ce se întâmplă când `if`-ul nu se declanșează **niciodată**: nu mai ai un produs, ai valoarea de pornire, iar ea iese pe ușă deghizată în răspuns.

La sumă nu te-a lovit niciodată, pentru că elementul neutru al adunării e **0**, iar **0** e și răspunsul corect pentru „nu există cifre pare”. La înmulțire cele două valori diferă — și exact acolo se rupe.

Interesant e că în **ex31** ai tratat cazul corect: acolo verifici `if (rezultat == 0)` și scrii „Nu sunt cifre pare”. Deci știi că situația există.

FIX: o funcție nu poate scrie mesaje, deci îi trebuie un semn distinct. Cel mai simplu e să numeri:

```
int produsulCifrelorPare(int numar) {
    int produsul = 1;
    int cate = 0;
    while (numar != 0) {
        int cifra = numar % 10;
        if (cifra % 2 == 0) {
            produsul = produsul * cifra;
            cate++;
        }
        numar = numar / 10;
    }
    if (cate == 0) {
        return 0;
    }
    return produsul;
}
```

CUM E ACUM: fişa [exercitii-cifre-functii-2.pdf](#) cere, cu exemple:

```
48723 , 1  ->  3
48723 , 3  ->  7
48723 , 9  -> -1
```

Codul tău dă:

```
cifraDePePozitie(48723, 1) = 2      cerut 3
cifraDePePozitie(48723, 3) = 8      cerut 7
cifraDePePozitie(48723, 9) = -1     corect
```

DE CE: algoritmul e bun — parcurgi cifrele de la dreapta, ții un contor, întorci cifra când contorul ajunge la poziția cerută, **-1** dacă numărul se termină înainte. Tot mecanismul e corect. Singurul lucru greșit e **de unde pornește contorul**: tu de la **0**, fișa numără de la **1**. Așa că fiecare răspuns e cu o cifră mai la stânga decât cel cerut.

Nu e o eroare de gândire, e o **convenție necită** — exact genul de lucru pentru care merită să rulezi exemplele din enunț înainte să declari exercițiul gata. Enunțul îți dăduse trei perechi de-a gata; prima te-ar fi prins.

FIX: `int ct = 1;` Restul funcției rămâne neschimbat.

B10 — zero e invizibil pentru buclele tale pe cifre

[functii.h:19](#) · [functii.h:90](#) · [functii.h:103](#) · [functii.h:172](#) · [functii.h:196](#) ·
[functii.h:222](#) · [exercitii-while.h:210](#) · [exercitii-while.h:262](#)

CUM E ACUM: fișa spune negru pe alb „numerele citite sunt pozitive **sau 0**”. Rulate cu **0**:

```

contorCifre(0)      -> 0      corect: 1
apareCifra(0, 0)    -> false  corect: true
cifraDePePozitie(0, 1) -> -1    corect: 0
ProdusulCifrelor(0) -> 1      corect: 0
ex14 cu n=0 -> "Numarul are: 0 cifre"      corect: 1
ex18 cu n=0 -> "Cifra minima este: 9"      corect: 0
contorAparitiiCifra(0, 0) -> 0      corect: 1
numarulDeCifre(0)    -> 0      corect: 1

```

Opt locuri, o singură cauză — și ultimele două le-ai scris azi, în aceeași zi în care ai pus prima gardă de zero din viața ta, în `cifreDiferite`.

Iar `numarulDeCifre(0) = 0` nu rămâne singur: `alipireNumire` îl folosește ca să știe cu cât să înmulțească, deci `alipireNumire(12, 0)` dă `12` în loc de `120`, iar `primaSiUltima(10)` dă `1` în loc de `10`. **Un bug într-o funcție de bază se duce în tot ce o cheamă** — de-aia se repară la rădăcină, nu în fiecare loc unde iese la suprafață.

DE CE: algoritmul „taie ultima cifră până nu mai rămâne nimic” folosește condiția `numar != 0` ca să știe când s-a terminat. Pentru `0`, condiția e falsă **din start** — bucla nu intră nici măcar o dată, iar funcția întoarce valoarea de pornire: `0` pentru contor, `false` pentru apariție, `1` pentru produs, `9` pentru minim.

Problema e că `0` joacă două roluri în același timp: e **numărul de prelucrat** și e **semnalul de oprire**. Când cele două coincid, bucla confundă „am terminat de tăiat” cu „n-am început”. Toate celelalte numere au undeva o cifră nenulă, deci scapă.

E același tipar ca la B8: valoarea de pornire iese pe ușă în locul unui rezultat.

FIX: tratează `0` separat, înainte de buclă:

```

int contorCifre(int numar) {
    if (numar == 0) {
        return 1;
    }
    ...
}

```

Apoi caută singur celelalte cinci și repară-le pe toate. Țasta e chiar exercițiul.

B11 — ex25 numără numerele, dar întreabă suma

exercitii-while.h:352-371

CUM E ACUM: rulat cu `5 -5 0` — două numere introduse:

Nu s-a introdus niciun numar

DE CE: ai o variabilă care știe exact ce te interesează — `numar`, pe care o crești la fiecare citire — și întrebi altceva. `suma == 0` nu înseamnă „n-a venit nimic”; înseamnă „ce a venit se anulează reciproc”. `5` și `-5` sunt două numere foarte reale a căror sumă e zero.

Un test se pune pe **mărimea care răspunde la întrebare**. Întrebarea e „câte?”, deci testul se pune pe contor. E același tipar ca la B10: o valoare (`0`) folosită pe post de semnal, deși poate apărea legitim ca dată.

Și dovada că știi regula: în `ex24`, scrisă chiar deasupra, ai pus `if (numar == 0)` — corect. Aceeași situație, două exerciții alăturate, două răspunsuri diferite.

FIX: `if ( numar == 0 )`

+ tot în `ex25`, linia 367: `int sumelor = n + suma;` — `n` e garantat `0` acolo, altfel bucla n-ar fi ieșit. Aduni zero la sumă. Scrie direct `suma`.

B12 — 12 uitat în mijlocul răspunsului (restanță din runda 1, B5)

operatii-conditionale.h:145

CUM E ACUM: rulat cu `x=13, y=23` → `UC(13+23)=126`. Ultima cifră a lui 36 e `6`.

DE CE: calculul e corect, `uc` chiar conține `6`. Greșeala e în **text**: în șirul `)=12"` au rămas două cifre dintr-un test vechi, lipite chiar înaintea rezultatului. Pentru `cout`, `)=12"` sunt doar caractere de tipărit, la fel ca `)="`. Le pune unul după altul, iar ochiul tău citește `126` ca pe un singur număr.

E un bug care supraviețuiește mult, pentru că nu strică nimic în cod — strică doar adevărul afișat. Singura plasă care îl prinde e să calculezi răspunsul pe hârtie **înainte** de a rula.

FIX: `cout << "UC(" << x << "+" << y << ")=" << uc << endl;`

B13 — proiectul tot nu se configurează pe altă mașină (restanță din runda 1, B1)

CMakeLists.txt:1

CUM E ACUM:

```
CMake Error at CMakeLists.txt:1 (cmake_minimum_required):
  CMake 4.3 or higher is required.  You are running version 4.2.3

-- Configuring incomplete, errors occurred!
```

Exact aceeași eroare ca acum trei săptămâni. Ca să-ți pot rula codul, l-am compilat pe lângă CMake, direct cu `g++`.

DE CE: `cmake_minimum_required` nu e o etichetă care spune „cu asta am lucrat eu”, e **un zid**: „nu continua dacă ai mai puțin”. CLion a pus acolo versiunea lui, cea mai nouă din lume, și a transformat-o în cerință minimă pentru oricine altcineva. La tine merge fiindcă tu ai chiar versiunea aia; la mine, la un coleg, pe un server de teme — nu.

Regula e invers decât pare: pui **cea mai mică** versiune care are ce-ți trebuie. Tu folosești `project()`, `set(CMAKE_CXX_STANDARD 20)` și `add_executable()` — trei lucruri vechi de zece ani.

FIX: `cmake_minimum_required(VERSION 3.20)`

B14 — `radacinaDigitala` se oprește la 10, care nu e cifră

`functii.h:318-327`

CUM E ACUM:

```
radacinaDigitala(123456) -> 3    corect
radacinaDigitala(199)    -> 10   gresit, trebuia 1
radacinaDigitala(19)     -> 10   gresit, trebuia 1
radacinaDigitala(28)     -> 10   gresit, trebuia 1
```

Rădăcina digitală e, prin definiție, **o cifră**. A ta întoarce 10.

```
while ( s > 10 ) {
    s = sumaCifrelor(s);
}
```

DE CE: condiția spune „mai strânge cât timp suma e **strict mai mare** decât 10”. Când suma ajunge exact la `10`, `10 > 10` e fals, bucla se oprește și `10` iese pe ușă ca răspuns. Îți lipsește exact un caracter: `>=`. Cu `>=`, la `10` bucla mai face o tură, `sumaCifrelor(10)` dă `1`, și abia ăla se oprește.

`10` e singura valoare la care se rupe — de-aia `123456` merge și te-a lăsat să crezi că e gata. Orice număr a cărui sumă de cifre atinge 10 pe drum cade: 19, 28, 37, 46, 55, 199, 145...

Și partea care contează cel mai mult din tot review-ul ăsta: **ai scris deja funcția asta, corect, acum o săptămână.** E `cifraDeControl`, la `functii.h:146`, și acolo ai scris `while (suma >= 10)`. Aceeași problemă, același fișier, două funcții — prima corectă, a doua greșită. Ai re-rezolvat-o alături în loc s-o refolosești, și la a doua încercare ai pierdut ce aveai la prima. (Vezi M3.)

FIX: `radacinaDigitala` nu trebuie rescrisă, trebuie ștearsă: `cifraDeControl` face deja exact asta. Dacă vrei numele nou, atunci:

```
int radacinaDigitala(int numar) {  
    return cifraDeControl(numar);  
}
```

B15 — `sumaCifrelorPePozitiiPare` nu face ce spune numele

`functii.h:259-271`

CUM E ACUM:

```
sumaCifrelorPePozitiiPare(8459) -> 12    pozitiile pare sunt 9 si 4, deci 13  
sumaCifrelorPare(8459)           -> 12    <- acelasi raspuns  
sumaCifrelorPePozitiiPare(1357) -> 0     pozitiile pare sunt 7 si 3, deci 10
```

```
int ct = 0;  
while ( numar > 0 ) {  
    int cifra = numar % 10;  
    if (cifra % 2 == 0) {        // <- cifra, nu ct  
        suma = suma + cifra;  
    }  
    numar = numar / 10;  
    ct++;  
}
```

DE CE: ai declarat `ct`, îl incrementezi la fiecare pas — și nu-l întrebi niciodată nimic. Testul se face pe `cifra`, adică funcția adună **cifrele pare**, nu cifrele **de pe poziții pare**. E o copie a lui

`sumaCifrelorPare` cu un contor lipit deasupra și un nume schimbat, de-aia dă exact același rezultat ca ea.

Compilerul ți-a spus-o, cuvânt cu cuvânt:

```
functii.h:261: warning: variable 'ct' set but not used
```

O variabilă pe care o scrii și n-o citești niciodată e aproape întotdeauna semnul că ai uitat să faci ceva cu ea. Merită să pornești `-Wall` în CLion și să citești warning-urile: sunt două în tot proiectul tău, și asta e un bug adevărat.

Și iar: **funcția de deasupra**, `sumaCifrelorDePePozitiiImpare`, e corectă — acolo ai scris `if (ct % 2 != 0)`. Ai scris perechea bună prima, apoi ai copiat-o și ai stricat-o. Pe `8459` dă 13, corect.

FIX:

```
if (ct % 2 == 0) {  
    suma = suma + cifra;  
}
```

🟡 Importante

M1 — 1601 din cele 1796 de linii ale tale nu ajung la compiler

`main.cpp:3`

`main.cpp` include un singur fișier: `functii.h`. `operatii-conditionale.h` (1133 linii) și `exercitii-while.h` (468 linii) nu sunt incluse de nimeni. Faptul că apar în `CMakeLists.txt` nu schimbă nimic — acolo fișierele `.h` sunt trecute doar ca să le vadă CLion în listă; compilerul nu le deschide decât dacă cineva le face `#include`.

Le-am verificat separat, cu `g++ -fsyntax-only` pe fiecare: **amândouă compilează**. Deci nu ai erori ascunse. Dar B6 a stat ascuns exact aici, o săptămână.

Ai făcut jumătatea grea din M5 (runda 1) — ai spart fișierul. Acum fă și cealaltă jumătate: adu-le înapoi în circuit.

```
#include "operatii-conditionale.h"
#include "exercitii-while.h"
#include "functii.h"

int main() {
    int optiune;
    cout << "Ce rulez? ";
    cin >> optiune;
    switch (optiune) {
        case 26: tema26(); break;
        case 32: ex32(); break;
    }
    return 0;
}
```

M2 — tema31 afișează sortarea de trei ori

operatii-conditionale.h:488-520

```
5 5 5 -> 5 5 5 / 5 5 5 / 5 5 5      de trei ori
3 3 7 -> 3 3 7 / 3 3 7              de doua ori
```

Logica de sortare e corectă. Problema sunt cele trei `if`-uri **independente**: pe `5 5 5`, toate trei condițiile (`a<=b && a<=c` , `b<=a && b<=c` , `c<=a && c<=b`) sunt adevărate deodată, fiindcă ai folosit `<=` . Aici întrebările chiar se exclud (un singur număr e „cel mai mic” pe care-l alegi), deci `else if` e forma corectă — invers decât la B2/B3, unde trebuiau `if`-uri separate. Regula e aceeași, aplicată în celălalt sens.

M3 — re-rezolvi alături în loc să reperi — de cinci ori

operatii-conditionale.h:1064 , :1022 , :1044 · functii.h:318 , :259 , :276

E tiparul cel mai costisitor al tău, și acum am cinci exemple:

varianta bună	varianta scrisă alături	ce s-a întâmplat
<code>tema12</code> (reparată)	<code>testtema12</code>	ambele corecte, cea veche greșită a rămas (<code>tema26</code>)
<code>tema51</code>	<code>tema52</code>	aceeași funcție de două ori, identice
<code>cifraDeControl</code>  <code>>=10</code>	<code>radacinaDigitala</code>  <code>>10</code>	a doua e greșită — B14
<code>sumaCifrelorDePePozitiiImpare</code> 	<code>sumaCifrelorPePozitiiPare</code> 	a doua e greșită — B15
<code>primaCifraa</code>  (tratează negativele)	rămâne nefolosită	<code>primaSiUltima</code> cheamă tot <code>primaCifra</code> — M12

Primele două erau doar dezordine. Ultimele trei sunt bug-uri: **când rescrii ceva ce ai scris deja bine, ai șanse reale să pierzi pe drum ce aveai.**

Când nimerești peste o greșeală, instinctul tău e să scrii varianta bună într-o funcție nouă. E un instinct bun pentru învățat — și prost pentru un proiect: codul greșit rămâne acolo, arată la fel de oficial ca cel bun, și peste o lună nu mai știi care e care. **Varianta nouă se scrie ca să înlocuiască, nu ca să stea alături.** Repară la sursă, apoi șterge dubura.

M4 — `contorCifrePare` e corectă, dar nu din motivul la care te gândești

`functii.h:31-40`

```
if ( numar % 2 == 0 ) { contor++; }
```

Peste tot în rest extragi cifra (`int cifra = numar % 10`) și abia apoi o testezi. Aici testezi direct `numar`. **Rezultatul e corect** — verificat pe `2468`, `13579`, `8459`, `105`.

Motivul e că un număr are aceeași paritate ca ultima lui cifră: `8459` e impar exact pentru că se termină în `9`. Deci `numar % 2` și `(numar % 10) % 2` dau mereu același lucru. E o scurtătură validă, dar una pe care ai nimerit-o, nu pe care ai ales-o — și care se rupe dacă întrebarea se schimbă (paritatea cifrei **de pe poziția a doua**, de pildă). Scrie-o ca pe restul.

M5 — aceeași buclă, două condiții de oprire

`functii.h:163` · `functii.h:174` · `functii.h:183`

`contorCifre` și `sumaCifrelor` se opresc pe `numar != 0`. `sumaCifrelorPare`, `ProdusulCifrelor` și `ProdusulCifrelorPare` se opresc pe `numar > 0`. Aceeași buclă, scrisă în două feluri în același fișier. Pe numere pozitive nu contează; pe `-123`, `contorCifre` întoarce `3` iar `ProdusulCifrelor` întoarce `1`. Alege una și ține-o peste tot.

M6 — `ex31` confundă „nu sunt cifre pare” cu „cifrele pare sunt toate 0”

`exercitii-while.h:439-459`

```
4738 -> Numarul par este: 48      corect
1357 -> Nu sunt cifre pare        corect
10    -> Nu sunt cifre pare        gresit, 0 e cifra para
```

Aceeași lecție ca la B8 și B10: folosești `0` ca semnal („n-am găsit nimic”), dar `0` poate apărea și ca dată reală. Un contor separat rezolvă și aici.

M7 — `tema10` nu acoperă toate notele (*restanță din runda 1, M1*)

`operatii-conditionale.h:254-273`

Nota `11` → nu afișează nimic. Cinci `if`-uri independente pentru 10, 9, 8, 7, 6, 5 și „sub 5”, niciun `else` pentru restul. Nota vine de la tastatură, deci poate fi orice. Ori un `else` care spune „notă invalidă”, ori o validare la început.

M8 — `tema28` convertește, nu descompune (*restanță din runda 1, M2*)

`operatii-conditionale.h:450-460`

`3661` secunde → „Minute: 61, Ore: 1”. Corect ar fi 1 oră, 1 minut, 1 secundă. `minute = secunde/60` dă **toate** minutele, nu cele rămase după ce ai scos orele. Ai formula corectă chiar la tine în fișier, la `tema39` :

```
int ore    = s / 3600;
int minute = (s % 3600) / 60;
int sec    = s % 60;
```

M9 — `tema5` împarte la zero (*restanță din runda 1, M3*)

operatii-conditionale.h:200

Cu `0` copii, `b / c` e împărțire la zero. La mine (Mac, ARM) a afișat liniștit `0`; pe Windows, pe Intel/AMD, același cod oprește programul cu eroare. Un `if (c == 0)` înainte de împărțire.

M10 — tema27 crede pe cuvânt că numărul are două cifre (restanță din runda 1, M4)

operatii-conditionale.h:439-448

Cu `123` afișează „Cifra zecilor este: 12”. Cerința spune două cifre, dar cerința nu ajunge la program — utilizatorul ajunge.

M11 — solutie1 scrie pe ecran dintr-o funcție

functii.h:48-58

Fișa cere explicit: „funcțiile întorc rezultatul, `cout` doar în `main()`”. `solutie1` citește de la tastatură și afișează. Celelalte 13 funcții respectă regula — asta e singura excepție, și e de fapt un `main` deghizat.

M12 — primaSiUltima cheamă versiunea veche, nu pe cea nouă

functii.h:292-294

```
primaSiUltima(4738) -> 48      corect
primaSiUltima(-4738) -> -47372 gresit
```

Ai scris `primaCifraa` (`functii.h:276`) special ca să trateze numerele negative — `if (numar < 0)` `numar = -numar;`. Și apoi, în `primaSiUltima`, chemi `primaCifra`, cea veche, fără tratarea aia. Versiunea nouă stă acolo nefolosită.

E a treia formă a aceluiași lucru din M3: scrii varianta bună alături și continui să folosești varianta veche. (+ `;;` la finalul liniei.)

M13 — alipireNumire folosește `pow` pentru o socoteală pe întregi

functii.h:231-233

```
return numar1 * pow(10, numarulDeCifre(numar2)) + numar2;
```

`pow` lucrează în virgulă mobilă și întoarce `double`; rezultatul e convertit înapoi la `int` la `return`. Merge pentru numerele tale, dar e unealta greșită: pentru a înmulți cu 10 de câteva ori nu-ți trebuie decât o buclă sau o înmulțire repetată, pe întregi curați.

Plus un amănunt de portabilitate: `pow` vine din `<cmath>`, pe care tu nu-l incluzi. La mine compilează fiindcă `<iostream>` îl trage după el; pe alt compilator (inclusiv unele de pe Windows) poate să nu compileze deloc.

M14 — exerciții din fișă încă nescrise

Din `exercitii-cifre-functii.pdf` / `-2.pdf` ți-au mai rămas trei: `cifraMaxima`, `cifraMinima` și „cifre strict crescătoare”. Le ai deja rezolvate ca exerciții cu `while` (`ex11`, `ex17`, `ex18`); e vorba doar să le muți în funcții care **întorc** rezultatul.

(`contorAparitii`, `ultimaCifra` și „cifre distincte” — bifate în `d9080ae`.)

● Cleanups

- **C1** — `functii.h:23`: `int cifra = numar % 10`; declarată și nefolosită în `contorCifre`. Unul din cele **două** warning-uri ale proiectului cu `-Wall -Wextra`; celălalt e `ct` din B15 și e un bug adevărat. Pornește-le în CLion.
- **C2** — `ProdusulCifrelor` și `ProdusulCifrelorPare` încep cu majusculă; celelalte 12 funcții încep cu literă mică. Convenția e literă mică: `produsulCifrelor`.
- **C3** — „bacnote” în loc de „bancnote”, de trei ori în `tema32`.
- **C4** — `inline` e pus doar la temele 32+, restul de 30 de funcții n-au. Nu deranjează acum, fiindcă header-ul e inclus dintr-un singur loc; în clipa în care apare al doilea `.cpp`, linker-ul se plânge de toate în afară de cele cu `inline`. Ori le pui la toate, ori la niciuna.
- **C5** — `tema29` (`operatii-conditionale.h:462`) are raza scrisă în cod (`double radius = 12`), nu citită, și folosește `std::cout` deși tot fișierul are `using namespace std`.
- **C6** — `operatii-conditionale.h:645`: `;;` — două puncte-și-virgulă.
- **C7** — `primasiultimaCifra(7)` întoarce `77`, fiindcă lipește prima cifră de ultima, iar la un număr de o cifră sunt aceeași. Nu e greșit, dar merită știut ce înseamnă rezultatul.
- **C8** — `exercitii-while.h:141`: `"Sunt " << contor << "numere pare"` → iese „Sunt 4numere pare”. Lipsește un spațiu.
- **C9** — `exercitii-while.h:196`: `int rest = n /= 10`; modifică `n` chiar în linia în care îl citești. Merge, dar se citește greu: `n = n / 10; int rest = n;`
- **C10** — `.idea/` (setările CLion) e tot comis în repo. Sunt fișiere ale calculatorului tău, nu ale proiectului.

- **C11** — commit-urile tale sunt tot semnate `"Todor <lucator54@gmail.com">`, cu ghilimele curbe intrate din greșeală. Se repară o dată: `git config --global user.name "Todor Luca"` și `git config --global user.email "lucator54@gmail.com"`.
- **C12** — două commit-uri se numesc „no message” (`707d870` , `02aabb6`). Mesajul de commit e singurul loc unde scrie ce ai făcut în ziua aia.
- **C13** — `CMakeLists.txt:9-10` : `functii.h` e trecut de două ori.
- **C14** — `main.cpp:4-8` : au rămas comentarii dintr-un exercițiu vechi (`//f(2)=7{` , `//x0y=xy+x+y`), plus cinci rânduri goale în `main`.

Before / After

#	Acum	Corect
B1	<code>cout</code> înăuntrul lui <code>else</code>	<code>cout</code> după <code>if/else</code> , pe drumul comun
B2	<code>if (bac50) { ... if (bac10) { ... } }</code>	<code>if (bac50) {...}</code> și <code>if (bac10) {...}</code> separate
B3	<code>else if (monede50) ... else if (moneda10)</code>	<code>if (monede50) ... if (moneda10) ...</code>
B4	<code>if (a > b && a > c)</code>	<code>if (a >= b && a >= c)</code> — la toate 3 ramurile
B5	<code>if (k < 0) { cout ... }</code> apoi calculul	<code>if (k < 0) {...} else { calculul }</code>
B6	<code>while (n != 0) { }</code>	bucula are nevoie de o linie care schimbă <code>n</code>
B7	<code>double media = double (suma / numar);</code>	<code>double media = (double) suma / numar;</code>
B8	<code>return produsul;</code>	<code>if (cate == 0) return 0;</code> apoi <code>return produsul;</code>
B9	<code>int ct = 0;</code>	<code>int ct = 1;</code>
B10	<code>while (numar != 0) { ... }</code>	<code>if (numar == 0) return 1;</code> înainte de buclă
B11	<code>if (suma == 0)</code>	<code>if (numar == 0)</code>
B12	<code>cout << ")=12" << uc;</code>	<code>cout << ")=" << uc;</code>
B13	<code>cmake_minimum_required(VERSION 4.3)</code>	<code>cmake_minimum_required(VERSION 3.20)</code>
B14	<code>while (s > 10)</code>	<code>while (s >= 10)</code> — sau <code>return cifraDeControl(numar);</code>

#	Acum	Corect
B15	<code>if (cifra % 2 == 0)</code>	<code>if (ct % 2 == 0)</code>

Q&A — răspunde-mi la astea înainte de runda 3

- La întrebarea 1 din runda 1 ai spus că **tăcerea** e mai periculoasă decât minciuna. Citește B4 și spune-mi dacă te-ai răzgândit — și dacă da, ce anume din explicație te-a convins. Dacă nu, argumentează: poate ai dreptate și eu greșesc.
- La `tema32` (bancomat) cele trei bancnote trebuie afișate cu `if` -uri **separate**. La `tema31` (sortare) cele trei cazuri trebuie puse pe `else if`. Amândouă au trei ramuri și amândouă sunt greșite acum. **Cum decizi, când scrii, care dintre cele două forme îți trebuie?** Dă-mi regula în cuvintele tale.
- `cifraDeControl` (scrisă acum o săptămână) și `radacinaDigitala` (scrisă azi) fac **exact același lucru**. Prima e corectă, a doua greșește pe 19, 28 și 199. Nu te întreb care e bug-ul — ăla ți l-am spus la B14. Te întreb: **ce te-a făcut s-o scrii a doua oară în loc s-o chemi pe prima?** Răspunsul ăsta mă interesează mai mult decât toate celelalte la un loc.
- Ai scris `ex32` cu corpul gol și n-ai observat o săptămână. Spune-mi **ce ai schimba în felul în care lucrezi** ca să nu se mai poată întâmpla — nu ce ai schimba în `ex32`.

Runda 1 — 2026-09-09 — sha `2cfec23`

Prima trecere prin `main.cpp` — 455 de linii, 28 de teme, toate rulate.

- **B1** `CMakeLists.txt` cere CMake 4.3 → ● **NEATINS** (runda 2, B13)
- **B2** `tema12` nu afișează nimic pe `5 5 3` → ✓ **ÎNCHIS** (`>=` pus corect)
- **B3** `tema26` afișează „Numerele sunt egale” pe `5 5 3` → ● **NEATINS** (runda 2, B4)
- **B4** `tema4` media notelor e `int` → ● **NEATINS**, plus o a doua apariție în `ex24` (runda 2, B7)
- **B5** `"=12"` lipit în fața rezultatului → ● **NEATINS** (runda 2, B12)
- **M1** `tema10` nu acoperă notele peste 10 → ● **NEATINS** (runda 2, M7)
- **M2** `tema28` convertește în loc să descompună → ● **NEATINS** (runda 2, M8)
- **M3** `tema5` împarte la zero → ● **NEATINS** (runda 2, M9)
- **M4** `tema27` nu validează numărul de două cifre → ● **NEATINS** (runda 2, M10)

- 🟡 **M5** 455 de linii într-un fișier, `main()` recomandat manual → 🟢 **PE JUMĂTATE**: fișierele sunt sparte în trei ✅, dar două din trei nu mai ajung la compilator și meniul lipsește (runda 2, M1)
- 🟢 **C1** variabilă nefolosită în `tema6` → ✅ închis, dar una nouă în `contorCifre`
- 🟢 **C2** teme fără `endl` la final → ✅ închis
- 🟢 **C3** indentare în trepte la `tema26` → ✅ închis
- 🟢 **C4** `.idea/` comis în repo → 🔴 **NEATINS**
- 🟢 **C5** autor git cu ghilimele curbe → 🔴 **NEATINS**
- 🟢 **C6** tabla înmulțirii de la 0 → ✅ închis (`ex5` pornește de la 1)
- **Q&A** → ✅ **răspuns în scris**, în `operatii-conditionale.h:1121-1132` . Răspunsul 2 e corect. Răspunsul 1 e pe dos — și explică de ce B3 a rămas deschis.

